

Trabalho 2 - Struct

Identificação do Aluno

- **Nome:** Rafael Lima Sant'Ana
- **Matrícula:** 251035659
- **Disciplina:** FGA0146 - ESTRUTURAS DE DADOS 1
- **Turma:** T04

3. Código-fonte Implementado (main.c)

O código abaixo reflete a engenharia completa da aplicação, com tratamento robusto de strings por meio do par `fgets()` e `sscanf()`, além do gerenciamento total de vazamento de memória:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
#include "tipos.h"

typedef struct NoI {
    TCadastroU *Dados;
    struct NoI *Prox;
} TNoI;

typedef struct NoC {
    TCadastroU *Dados;
    struct NoC *Ant;
    struct NoC *Prox;
} TNoC;

typedef struct NoS {
    TCadastroU *Dados;
    struct NoS *Ant;
    struct NoS *Prox;
} TNoS;

typedef struct Descritores {
    TNoS *Inicio;
```

```

    int Tamanho;
    TNoS *Fim;
} TDescritoS;

typedef struct NoU {
    TCadastroU *Dados;
    struct NoU *Ant;
    struct NoU *Prox;
} TNoU;

typedef struct DescritorU {
    TNoU *Inicio;
    int Tamanho;
    TNoU *Fim;
} TDescritorU;

typedef struct ResultadosEntrada {
    TNoI *Industria;
    TNoC *Comercio;
} TResultadosEntrada;

TNoI* inserirIndustria(TNoI *topo, TCadastroU *dados);
TNoC* inserirComercio(TNoC *topo, TCadastroU *dados);
void inserirServico(TDescritoS *desc, TCadastroU *dados);
void inserirUnificada(TDescritorU *desc, TCadastroU *dados, char
tipo);

TResultadosEntrada carregarDadosEntrada(TNoI *listaI, TNoC *listaC,
TDescritoS *listaS);
void gerarListaUnificada(TNoI *listaI, TNoC *listaC, TDescritoS
*listaS, TDescritorU *listaU);

void relatorioIndustrias(TNoI *listaI);
void relatorioComercio(TNoC *listaC);
void relatorioComercioInvertido(TNoC *listaC);
void relatorioServicos(TDescritoS *listaS);
void relatorioServicosInvertido(TDescritoS *listaS);
void relatorioUnificada(TDescritorU *listaU);
void relatorioUnificadaInvertido(TDescritorU *listaU);

void apagarListas(TNoI **listaI, TNoC **listaC, TDescritoS
*listaS, TDescritorU *listaU);
void imprimirCadastro(TCadastroU *c);

```

```

int main() {
    TNoI *listaIndustria = NULL;
    TNoC *listaComercio = NULL;
    TDescriptorS listaServicos = {NULL, 0, NULL};
    TDescriptorU listaUnificada = {NULL, 0, NULL};

    char buffer[50];
    int opcao = -1;

    while (opcao != 0) {
        printf("\n===== MENU PRINCIPAL\n");
        printf("1. Carregar Dados de Entrada\n");
        printf("2. Gerar Lista Unificada\n");
        printf("3. Relatorio: Industrias\n");
        printf("4. Relatorio: Comercio\n");
        printf("5. Relatorio: Comercio Invertida\n");
        printf("6. Relatorio: Servicos\n");
        printf("7. Relatorio: Servicos Invertida\n");
        printf("8. Relatorio: Lista Unificada\n");
        printf("9. Relatorio: Lista Unificada Invertida\n");
        printf("10. Apagar Listas\n");
        printf("0. Sair\n");
        printf("Escolha uma opcao: ");

        if (fgets(buffer, sizeof(buffer), stdin) != NULL) {
            if (sscanf(buffer, "%d", &opcao) != 1) {
                opcao = -1;
            }
        }

        switch (opcao) {
            case 1: {
                TResultadosEntrada res =
carregarDadosEntrada(listaIndustria, listaComercio,
&listaServicos);
                listaIndustria = res.Industria;
                listaComercio = res.Comercio;
                break;
            }
            case 2:
                gerarListaUnificada(listaIndustria, listaComercio,

```

```

&listaServicos, &listaUnificada);
    break;
    case 3:
        relatorioIndustrias(listaIndustria);
        break;
    case 4:
        relatorioComercio(listaComercio);
        break;
    case 5:
        relatorioComercioInvertido(listaComercio);
        break;
    case 6:
        relatorioServicos(&listaServicos);
        break;
    case 7:
        relatorioServicosInvertido(&listaServicos);
        break;
    case 8:
        relatorioUnificada(&listaUnificada);
        break;
    case 9:
        relatorioUnificadaInvertido(&listaUnificada);
        break;
    case 10:
        apagarListas(&listaIndustria, &listaComercio,
&listaServicos, &listaUnificada);
        break;
    case 0:
        apagarListas(&listaIndustria, &listaComercio,
&listaServicos, &listaUnificada);
        printf("Saindo do programa. Ate logo!\n");
        break;
    default:
        printf("Opcao invalida! Tente novamente.\n");
        break;
}
}
return 0;
}

```

```

TNoI* inserirIndustria(TNoI *topo, TCadastroU *dados) {
    TNoI *novo = (TNoI*) malloc(sizeof(TNoI));
    novo->Dados = dados;
}

```

```

    novo->Prox = topo;
    return novo;
}

```

```

TNoC* inserirComercio(TNoC *topo, TCadastroU *dados) {
    TNoC *novo = (TNoC*) malloc(sizeof(TNoC));
    novo->Dados = dados;
    novo->Ant = NULL;
    novo->Prox = topo;

    if (topo != NULL) {
        topo->Ant = novo;
    }
    return novo;
}

```

```

void inserirServico(TDescriptorS *desc, TCadastroU *dados) {
    TNoS *novo = (TNoS*) malloc(sizeof(TNoS));
    novo->Dados = dados;
    novo->Prox = NULL;
    novo->Ant = desc->Fim;

    if (desc->Inicio == NULL) {
        desc->Inicio = novo;
        desc->Fim = novo;
    } else {
        desc->Fim->Prox = novo;
        desc->Fim = novo;
    }
    desc->Tamanho++;
}

```

```

void inserirUnificada(TDescriptorU *desc, TCadastroU *dados, char
tipo) {
    TNoU *novo = (TNoU*) malloc(sizeof(TNoU));

    novo->Dados = (TCadastroU*) malloc(sizeof(TCadastroU));
    memcpy(novo->Dados, dados, sizeof(TCadastroU));
    novo->Dados->Tipo = tipo;

    novo->Prox = NULL;
    novo->Ant = desc->Fim;
}

```

```

    if (desc->Inicio == NULL) {
        desc->Inicio = novo;
        desc->Fim = novo;
    } else {
        desc->Fim->Prox = novo;
        desc->Fim = novo;
    }
    desc->Tamanho++;
}

TResultadosEntrada carregarDadosEntrada(TNoI *listaI, TNoC *listaC,
TDescriptorS *listaS) {
    FILE *arq;
    TResultadosEntrada resultado;
    char linha[150];

    listaS->Inicio = NULL;
    listaS->Fim = NULL;
    listaS->Tamanho = 0;

    arq = fopen("Industria.txt", "r");
    if (arq != NULL) {
        while (fgets(linha, sizeof(linha), arq) != NULL) {
            TCadastroU *cad = (TCadastroU*)
malloc(sizeof(TCadastroU));
            if (sscanf(linha, "%14s %49[^0-9] %29s %11s",
cad->CNPJ, cad->RazaoSocial, cad->Cidade, cad->Fone) >= 3) {
                listaI = inserirIndustria(listaI, cad);
            } else {
                free(cad);
            }
        }
        fclose(arq);
        printf("Dados de Industria.txt carregados com sucesso.\n");
    } else {
        printf("Aviso: Arquivo Industria.txt nao encontrado.\n");
    }

    arq = fopen("Comercio.txt", "r");
    if (arq != NULL) {
        while (fgets(linha, sizeof(linha), arq) != NULL) {
            TCadastroU *cad = (TCadastroU*)
malloc(sizeof(TCadastroU));

```

```

        if (sscanf(linha, "%14s %49[^0-9] %29s %11s",
cad->CNPJ, cad->RazaoSocial, cad->Cidade, cad->Fone) >= 3) {
            listaC = inserirComercio(listaC, cad);
        } else {
            free(cad);
        }
    }
    fclose(arq);
    printf("Dados de Comercio.txt carregados com sucesso.\n");
} else {
    printf("Aviso: Arquivo Comercio.txt nao encontrado.\n");
}

```

```

    arq = fopen("Servico.txt", "r");
    if (arq != NULL) {
        while (fgets(linha, sizeof(linha), arq) != NULL) {
            TCadastroU *cad = (TCadastroU*)
malloc(sizeof(TCadastroU));
            if (sscanf(linha, "%14s %49[^0-9] %29s %11s",
cad->CNPJ, cad->RazaoSocial, cad->Cidade, cad->Fone) >= 3) {
                inserirServico(listaS, cad);
            } else {
                free(cad);
            }
        }
        fclose(arq);
        printf("Dados de Servico.txt carregados com sucesso.\n");
    } else {
        printf("Aviso: Arquivo Servico.txt nao encontrado.\n");
    }
}

```

```

    resultado.Industria = listaI;
    resultado.Comercio = listaC;
    return resultado;
}

```

```

void gerarListaUnificada(TNoI *listaI, TNoC *listaC, TDescriptorS
*listaS, TDescriptorU *listaU) {
    if (listaU->Inicio != NULL) {
        printf("A lista unificada ja contem elementos. Apague-a
antes de gerar novamente.\n");
        return;
    }
}

```

```

    TNoI *atualI = listaI;
    while (atualI != NULL) {
        inserirUnificada(listaU, atualI->Dados, 'I');
        atualI = atualI->Prox;
    }

    TNoC *atualC = listaC;
    while (atualC != NULL) {
        inserirUnificada(listaU, atualC->Dados, 'C');
        atualC = atualC->Prox;
    }

    TNoS *atualS = listaS->Inicio;
    while (atualS != NULL) {
        inserirUnificada(listaU, atualS->Dados, 'S');
        atualS = atualS->Prox;
    }

    printf("Lista Unificada gerada com sucesso! Total de registros:
%d\n", listaU->Tamanho);
}

void imprimirCadastro(TCadastroU *c) {
    printf("CNPJ: %-15s | Razao Social: %-25s | Cidade: %-15s |
Fone: %-12s",
        c->CNPJ, c->RazaoSocial, c->Cidade, c->Fone);
    if (c->Tipo == 'I' || c->Tipo == 'C' || c->Tipo == 'S') {
        printf(" | Tipo: %c", c->Tipo);
    }
    printf("\n");
}

void relatorioIndustrias(TNoI *listaI) {
    printf("\n--- RELATORIO: INDUSTRIAS ---\n");
    if (listaI == NULL) printf("Lista vazia.\n");
    while (listaI != NULL) {
        imprimirCadastro(listaI->Dados);
        listaI = listaI->Prox;
    }
}

void relatorioComercio(TNoC *listaC) {

```



```

    printf("\n--- RELATORIO: COMERCIO ---\n");
    if (listaC == NULL) printf("Lista vazia.\n");
    while (listaC != NULL) {
        imprimirCadastro(listaC->Dados);
        listaC = listaC->Prox;
    }
}

void relatorioComercioInvertido(TNoC *listaC) {
    printf("\n--- RELATORIO: COMERCIO INVERTIDO ---\n");
    if (listaC == NULL) {
        printf("Lista vazia.\n");
        return;
    }
    while (listaC->Prox != NULL) {
        listaC = listaC->Prox;
    }
    while (listaC != NULL) {
        imprimirCadastro(listaC->Dados);
        listaC = listaC->Ant;
    }
}

void relatorioServicos(TDescritoS *listaS) {
    printf("\n--- RELATORIO: SERVICOS (Tamanho: %d) ---\n",
listaS->Tamanho);
    TNoS *atual = listaS->Inicio;
    if (atual == NULL) printf("Lista vazia.\n");
    while (atual != NULL) {
        imprimirCadastro(atual->Dados);
        atual = atual->Prox;
    }
}

void relatorioServicosInvertido(TDescritoS *listaS) {
    printf("\n--- RELATORIO: SERVICOS INVERTIDO ---\n");
    TNoS *atual = listaS->Fim;
    if (atual == NULL) printf("Lista vazia.\n");
    while (atual != NULL) {
        imprimirCadastro(atual->Dados);
        atual = atual->Ant;
    }
}

```

```

void relatorioUnificada(TDescritorU *listaU) {
    printf("\n--- RELATORIO: LISTA UNIFICADA (Tamanho: %d) ---\n",
listaU->Tamanho);
    TNoU *atual = listaU->Inicio;
    if (atual == NULL) printf("Lista vazia.\n");
    while (atual != NULL) {
        imprimirCadastro(atual->Dados);
        atual = atual->Prox;
    }
}

```

```

void relatorioUnificadaInvertido(TDescritorU *listaU) {
    printf("\n--- RELATORIO: LISTA UNIFICADA INVERTIDA ---\n");
    TNoU *atual = listaU->Fim;
    if (atual == NULL) printf("Lista vazia.\n");
    while (atual != NULL) {
        imprimirCadastro(atual->Dados);
        atual = atual->Ant;
    }
}

```

```

void apagarListas(TNoI **listaI, TNoC **listaC, TDescritors
*listaS, TDescritorU *listaU) {
    TNoI *atualI = *listaI;
    while (atualI != NULL) {
        TNoI *aux = atualI->Prox;
        free(atualI->Dados);
        free(atualI);
        atualI = aux;
    }
    *listaI = NULL;

    TNoC *atualC = *listaC;
    while (atualC != NULL) {
        TNoC *aux = atualC->Prox;
        free(atualC->Dados);
        free(atualC);
        atualC = aux;
    }
    *listaC = NULL;

```

```

    TNoS *atuais = listaS->Inicio;

```

```
    while (atualS != NULL) {
        TNoS *aux = atualS->Prox;
        free(atualS->Dados);
        free(atualS);
        atualS = aux;
    }
    listaS->Inicio = NULL;
    listaS->Fim = NULL;
    listaS->Tamanho = 0;

    TNoU *atualU = listaU->Inicio;
    while (atualU != NULL) {
        TNoU *aux = atualU->Prox;
        free(atualU->Dados);
        free(atualU);
        atualU = aux;
    }
    listaU->Inicio = NULL;
    listaU->Fim = NULL;
    listaU->Tamanho = 0;

    printf("\nTodas as estruturas de dados foram desalocadas e
limpas da memoria com sucesso.\n");
}
```